# Essential Python Topics for Selenium

Md. Ehsanul Haque
Senior Software Quality Assurance Engineer
Vivasoft Ltd.

# Agenda

This session will provide an in-depth understanding of variables, data types and operators in Python.

- **Python Basics**
  - What is Python?
  - Identifiers
  - Keywords
  - Variables and data types (str, int, float, bool)
  - Operators (arithmetic, logical, comparison)
- **Data Structures**
  - List
  - Dictionaries
  - Tuples
  - Sets

# What is an identifier in Python?

An **identifier** is the name used to identify a variable, function, class, module, or object.

```
name = "Alice"
def greet():
    pass
class Student:
    pass
```

In the above Python code, we have few identifiers as follows:

- **name**: Variable Name
- **greet**: Function Name
- **Student**: Class Name

# What is Python?

**Key Features of Python:**

- **Easy to read and write** — Clean, English-like syntax
- **Interpreted** — Runs directly without needing compilation
- **Dynamically typed** — No need to declare variable types
- **Object-oriented** — Supports classes and objects
- **Extensive standard library** — Built-in support for web, math, file handling, etc.
- **Cross-platform** — Runs on Windows, macOS, Linux
- **Huge community and ecosystem** — Libraries for data science, automation, web development, etc.

**What Is Python Used For?**

- **Web development** – using frameworks like Django, Flask
- **Automation / Scripting** – test automation, task automation
- **Data science & AI** – using libraries like NumPy, Pandas, TensorFlow
- **Software development**
- **Desktop applications**
- **Cybersecurity and Ethical Hacking**

# Rules for Naming Identifiers

- Must begin with:
  - a letter (A–Z, a–z) or underscore (_)
- Can contain:
  - Letters
  - Digits (0–9)
  - Underscores
- Cannot begin with a digit
- Cannot be a Python keyword
- Are case-sensitive (name, Name, and NAME are different)
- Cannot contain special characters (@, #, !, -, etc.)
- Should not use built-in function names (e.g., list, input)
- Can be of any length

## Valid Identifiers

| Identifier | Reason |
|---|---|
| `name` | Letters only |
| `name1` | Letters and digits |
| `_value` | Starts with underscore |
| `user_name` | Letters and underscore |
| `A1B2C3` | Letters and digits |
| `studentAge` | Camel case, valid |
| `__init__` | Special method name (valid) |

## Invalid Identifiers

| Identifier | Reason |
|---|---|
| `1name` | Starts with a digit |
| `user-name` | Contains a hyphen `-` (invalid char) |
| `@user` | Contains `@` (invalid char) |
| `class` | Python keyword |
| `def` | Python keyword |
| `import` | Python keyword |
| `first name` | Contains a space |

# What is a variable?

A variable in Python is a named reference to a value stored in memory. It allows you to store, retrieve, and manipulate data throughout your program.

- Think of a variable as a labeled box in memory that holds something (a value).

# Variable Declaration and Assignment

Python uses the **assignment operator** = to assign values to variables.

```
x = 10       # int
name = "Eva" # str
```

Python is **dynamically typed**, which means you don't need to declare the type beforehand.

## Types of Variables

- by value stored
  - Which type of data stored
- by scope
  - Where the variable declared

## Types of Variables (by value stored)

| Type | Description | Example |
|---|---|---|
| int | Integer numbers | x = 42 |
| float | Decimal numbers | pi = 3.14 |
| str | Text/strings | name = "Alice" |
| bool | Boolean values | is_active = True |
| list | Ordered, mutable sequence | nums = [1, 2, 3] |
| tuple | Ordered, immutable sequence | t = (1, 2, 3) |
| dict | Key-value pairs | d = {"a": 1} |
| set | Unordered collection of unique values | s = {1, 2, 3} |
| NoneType | Represents absence of value | x = None |

# Types of Variables (by scope)

- Local
- Global
- Nonlocal

- **Local Variable**
  - Declared **inside a function**.
  - Accessible **only within that function**.
  - Automatically destroyed when the function ends.
- **Global Variable**
  - Declared **outside of all functions**.
  - Accessible **anywhere** in the script (inside and outside functions).
  - To modify it inside a function, use the global keyword.
- **Nonlocal**
  - Used **in nested functions**.
  - Refers to a variable in the **nearest enclosing (but not global) scope**.
  - Declared using nonlocal.

## Python is Dynamically and strongly typed.

- **Dynamically Typed**: No need to declare variable types.
- **Strongly Typed**: Type safety is enforced; incompatible types cause errors.

```
x = 5

x = "hello"  # Okay (dynamic typing)

y = 10 + "5"  # Error (can't add int + str)
```

## Multiple Assignment

You can assign multiple variables in one line:

```
x, y, z = 1, 2, 3
```

Or assign the same value to multiple variables:

```
a = b = c = 100
```

**Query:**

- Variable Naming Rules

# Type Casting in Python

- **Implicit Casting (Automatic)**
  - Python automatically converts one data type to another **without the programmer's involvement**.
- **Explicit Casting (Narrowing)**
  - **Manually convert** one data type to another using built-in functions.

## Common Type Casting Functions:

| Function | Converts To | Example |
|---|---|---|
| int() | Integer | int("10") → 10 |
| float() | Floating point | float("5.5") → 5.5 |
| str() | String | str(100) → "100" |
| bool() | Boolean | bool(1) → True |
| list() | List | list("abc") → ['a', 'b', 'c'] |
| tuple() | Tuple | tuple([1, 2]) → (1, 2) |
| set() | Set | set("aa") → {'a'} |
| dict() | Dictionary | dict([(1, "a")]) → {1: "a"} |

## Invalid Conversions (raise errors)

```
int("abc")      # ❌ ValueError
float("1.2.3") # ❌ ValueError
int([1, 2, 3]) # ❌ TypeError
```

# Python Operators

Operators in python are symbols that perform operations on variables and values.

## Python supports different types of operators:

- **Arithmetic operators:** These operators are used to perform basic arithmetic operations.
- **Logical Operators:** Used to combine multiple conditions.
- **Assignment Operator:** Used to assign values to variables.
- **Comparison Operators:** Compare values
- Etc...

# Python Operators in Selenium

## 1. Arithmetic Operators in Selenium

- **Scenario: Verify price calculation in a shopping cart**

## 2. Logical Operators in Selenium

- **Scenario: Click “Buy Now” only if user is logged in and item is in stock**

## 3. Assignment Operator in Selenium

- **Scenario: Store scraped product name into a variable**

## 4. Comparison Operators in Selenium

- **Scenario: Validate discount applied**

# Data Structures

- **Lists**
  - A list is a collection of **ordered**, **mutable**, and **indexed** elements.
  - (indexing, slicing, list methods)
- **Dictionaries**
  - A dictionary is an **unordered**, **mutable** collection of key-value pairs.
  - (keys, values, items)
- **Tuples**
  - A tuple is a collection of **ordered**, **immutable** elements.
- **Sets**
  - collection of unordered, unique, and mutable elements.

# Lists

- **A list is a collection of ordered, mutable, and indexed elements.**
- **(indexing, slicing, list methods)**

**Creating a List:**

- fruits = ['apple', 'banana', 'cherry']

**Indexing:**

- Accessing elements using index (starts from 0).
- print(fruits[0])  # Output: 'apple'

**Slicing:**

- Extracting parts of a list.
- print(fruits[1:3])  # Output: ['banana', 'cherry']
- print(fruits[:2])  # Output: ['apple', 'banana']
- print(fruits[-1])  # Output: 'cherry'

# Common List Methods:

| Method | Description | Example |
|---|---|---|
| `append(x)` | Adds an item to the end of the list. | `my_list.append(10)` |
| `extend(iterable)` | Adds all items from an iterable (like another list). | `my_list.extend([4, 5])` |
| `insert(i, x)` | Inserts item `x` at index `i`. | `my_list.insert(1, 20)` |
| `remove(x)` | Removes the first occurrence of item `x`. | `my_list.remove(3)` |
| `pop([i])` | Removes and returns item at index `i` (last item if index not specified). | `my_list.pop()` |
| `clear()` | Removes all items from the list. | `my_list.clear()` |
| `index(x)` | Returns the index of the first occurrence of `x`. | `my_list.index(3)` |
| `count(x)` | Returns the number of times `x` appears. | `my_list.count(2)` |
| `sort()` | Sorts the list in ascending order (in-place). | `my_list.sort()` |
| `reverse()` | Reverses the list in-place. | `my_list.reverse()` |
| `copy() or list()` | Returns a shallow copy of the list. | `new_list = my_list.copy()`<br>`mylist = list(thislist)` |

# Dictionaries

- **A dictionary is an unordered, mutable collection of key-value pairs.**
- **(keys, values, items)**

**Creating a Dictionary:**

- student = {'name': 'Alice', 'age': 23, 'grade': 'A'}

**Accessing Values:**

- print(student['name'])  # Output: 'Alice'
- print(student.get('age'))  # Output: 23

# Common Dictionary Methods:

| Method | Description | Example |
|---|---|---|
| `dict.get(key, default)` | Returns the value for the key if it exists, else returns default (None if not provided). | `my_dict.get("name", "Unknown")` |
| `dict.keys()` | Returns a view object of all keys. | `my_dict.keys()` |
| `dict.values()` | Returns a view object of all values. | `my_dict.values()` |
| `dict.items()` | Returns a view object of key-value pairs. | `my_dict.items()` |
| `dict.update(other_dict)` | Updates the dictionary with key-value pairs from another dictionary. | `my_dict.update({"age": 25})` |
| `dict.pop(key, default)` | Removes the specified key and returns the value. Returns default if key not found. | `my_dict.pop("name", "Not found")` |
| `dict.popitem()` | Removes and returns the last inserted key-value pair (LIFO). | `my_dict.popitem()` |
| `dict.clear()` | Removes all items from the dictionary. | `my_dict.clear()` |
| `dict.copy()` | Returns a shallow copy of the dictionary. | `new_dict = my_dict.copy()` |

# Tuples

- **A tuple is a collection of ordered, immutable elements.**

**Creating a Tuple:**

- coordinates = (10.5, 20.3)

**Accessing Tuple Elements:**

- print(coordinates[0])  # Output: 10.5
- Use tuples when the data should **not change**.

# Sets

- **A set is a collection of unordered, unique, and mutable elements.**

**Creating a Set:**

- colors = {'red', 'green', 'blue'}

**Set Operations:**

- colors.add('yellow') # Adds an element
- colors.remove('green') # Removes an element
- print('blue' in colors) # Membership test

# Common Set Methods:

| Method | Description | Example |
|---|---|---|
| `add(x)` | Adds an element `x` to the set. | `my_set.add(10)` |
| `update(iterable)` | Adds elements from an iterable. | `my_set.update([2, 3])` |
| `remove(x)` | Removes element `x`; raises `KeyError` if not found. | `my_set.remove(3)` |
| `discard(x)` | Removes element `x`; does nothing if not found. | `my_set.discard(3)` |
| `pop()` | Removes and returns an arbitrary element. | `my_set.pop()` |
| `clear()` | Removes all elements from the set. | `my_set.clear()` |
| `union(set2)` | Returns a new set with elements from both sets. | `a.union(b)` |
| `intersection(set2)` | Returns common elements of both sets. | `a.intersection(b)` |
| `difference(set2)` | Elements in `a` but not in `b`. | `a.difference(b)` |
| `issubset(set2)` | Checks if set is subset of another. | `a.issubset(b)` |
| `issuperset(set2)` | Checks if set is superset of another. | `a.issuperset(b)` |
| `isdisjoint(set2)` | Checks if two sets have no elements in common. | `a.isdisjoint(b)` |
| `copy()` | Returns a shallow copy of the set. | `b = a.copy()` |

# Overview & Characteristics

| Data Structure | Ordered | Mutable | Duplicate Allowed | Use Case in Selenium |
|---|---|---|---|---|
| **List** | Yes | Yes | Yes | Storing and processing multiple elements (e.g., web elements) |
| **Tuple** | Yes | No | Yes | Fixed data that shouldn't change (e.g., locator strategies) |
| **Dictionary** | No | Yes | Keys unique | Mapping keys to values (e.g., element names to locators) |
| **Set** | No | Yes | No | Unique elements (e.g., filtering duplicates from a list of results) |

# Lists in Selenium Automation

In **Selenium automation**, lists in Python are commonly used to:

- Store multiple web elements (e.g., list of buttons, rows, checkboxes)
- Iterate through and interact with them (click, extract text, validate, etc.)
- Handle dynamic web content
- Collect data for assertions

## 1. Storing Multiple Elements

- **Scenario:** Click all checkboxes on a page.

## 2. Extracting Text from a List of Elements

- **Scenario:** Get all product names from a product list.

## 3. Validating Table Column Values

- **Scenario:** Validate that all prices in a column are under $100.

## 4. Handling Dropdown Options

- **Scenario:** Print all options from a dropdown.

## 5. Conditional Action Based on List Contents

- **Scenario:** Click the button next to the product named "Laptop".

# Dictionaries in Selenium Automation

In **Selenium automation**, Python dictionaries (`dict`) are very useful for:

- Mapping values (e.g., labels to locators)
- Organizing test data for data-driven testing
- Storing results and assertions
- Handling configurations and page elements dynamically

### 1. Storing and Verifying Output Results

- **Scenario:** Validate product info on a product detail page.

### 2. Data-Driven Testing with Test Data Dictionary

- **Scenario:** Fill out a form using key-value pairs.

### 3. Storing Page URLs or Routes

- **Scenario:** Navigate to specific pages using dictionary mapping.

### 4. Storing Test Case Status or Logs

- **Scenario:** Record pass/fail status of multiple tests.

# Tuples in Selenium Automation

In **Selenium automation**, **tuples** in Python are often used when you need:

- Immutable pairs (e.g., locator strategies like `(By.ID, "value")`)
- Storing fixed pairs of data (e.g., field name and input)
- Returning multiple values from a function

### 1. Locators as Tuples

- **Scenario:** Store locators centrally and reuse them.

### 2. Returning Multiple Values from a Function

- **Scenario:** Create a utility to fetch both element and its text.

### 3. Handling Multi-Locator Strategies

- **Scenario:** Try a list of locators in order until one works.

# Sets in Selenium Automation

In **Selenium automation**, Python **sets** can be useful for:

- Handling **unique values** (e.g., unique text from elements)
- Comparing sets of data (e.g., expected vs. actual results)
- Removing duplicates from lists of elements
- Verifying items across pages or tables

Sets are **unordered** collections that **do not allow duplicates**, which makes them ideal for verification and comparison tasks.

### 1. Compare Expected vs. Actual Values

- **Scenario:** Validate that all required menu items are present on the page.

### 2. Find Common Items Between Two Lists (Intersection)

- **Scenario:** Compare items in two sections and find duplicates.

### 3. Check if Two Pages Show the Same Set of Elements

- **Scenario:** Compare product names from two different pages.

# Agenda

This session will provide an in-depth understanding of control structures in Python.

- **Control Structures**
  - if, elif, else
  - for and while loops
  - break and continue

# Control Structures in Python

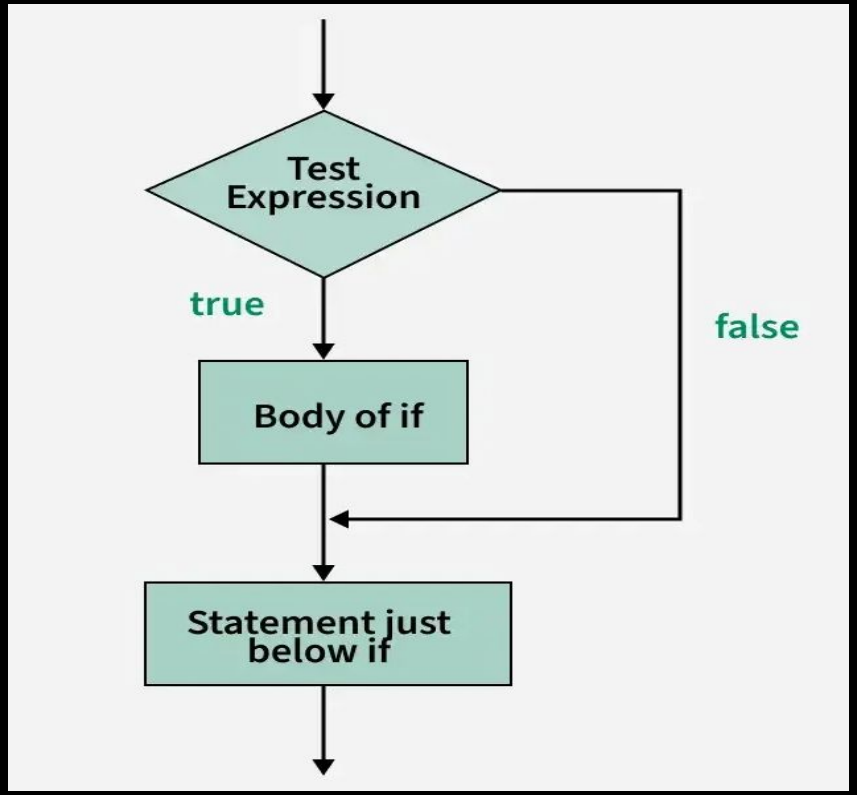
| Structure | Use Case | Python Version |
|---|---|---|
| if, else | Decision-making | All versions |
| for | Repeating actions | All versions |
| break, continue | Loop flow control | All versions |

- **Conditional Statements**
  - if statement
  - if...else statement
  - if...elif...else statement
- **Looping Statements**
  - for loop
  - while loop
- **Loop Control Statements**
  - Break
  - Continue

# **Conditional Statements** in Python

- if statement
- if...else statement
- if...elif...else statement

## If Statements in Python



# Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

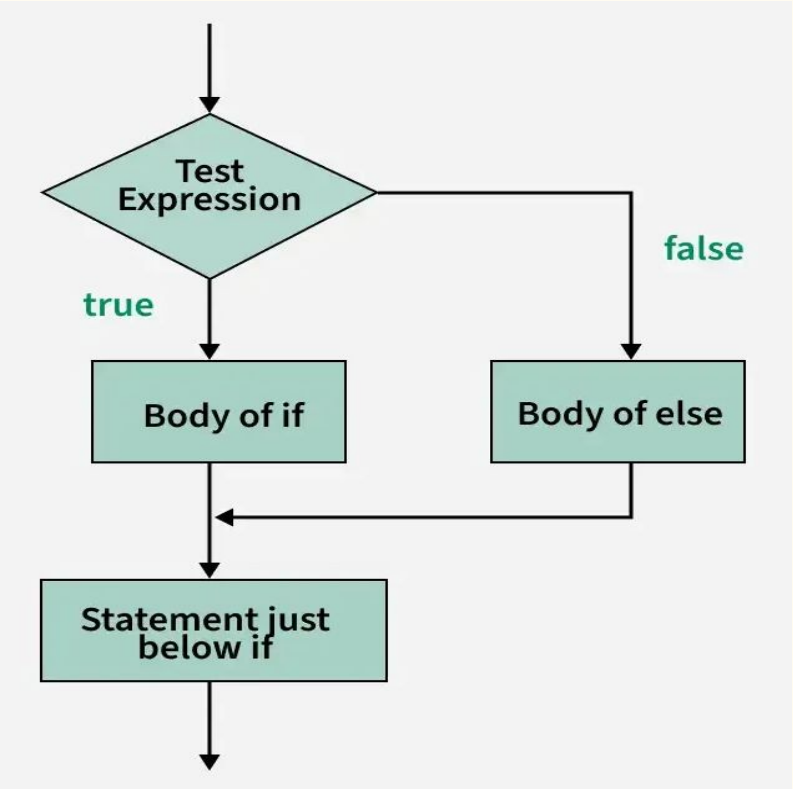
These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the if keyword.

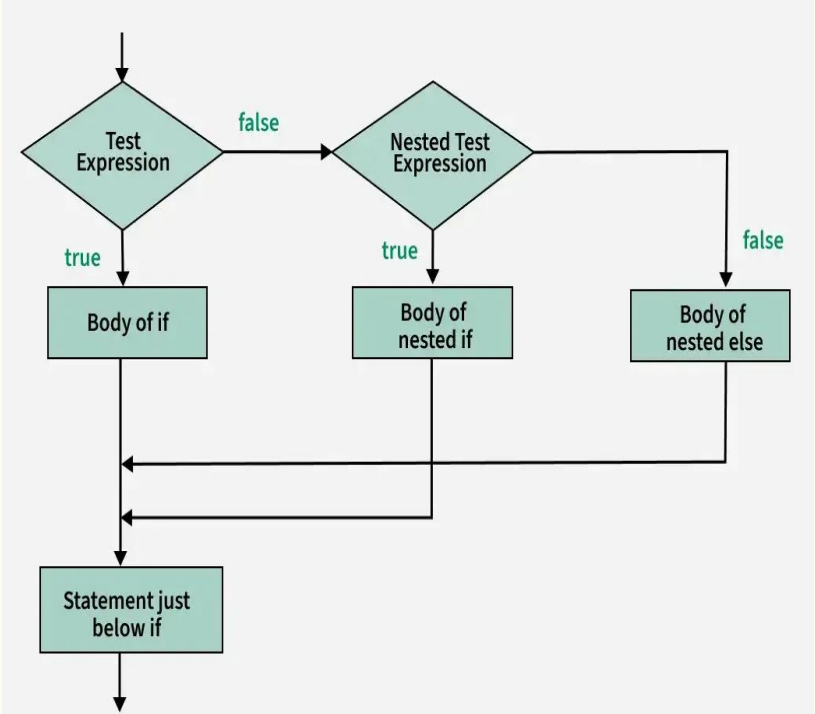
```
x = 10
if x > 0:
    print("Positive")
```

- In this example we use one variables x, which is used as part of the if statement to test whether x is greater than 0. As x is 10, we know that 10 is greater than 0, and so we print to screen that "Positive".
- Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

# If else Statements




# if...elif...else statement



# Python Conditions and if-else statements

The else keyword catches anything which isn't caught by the preceding conditions.

```
# Check whether a number is positive or negative

x = -1  # Hardcoded value

# Decision making using if-else
if x > 0:
    print("The number is positive.")
else:
    print("The number is negative.")
```

In this example we use one variables x, which is used as part of the if statement to test whether x is greater than 0. As x is -1, we know that -1 is not greater than 0, so the first condition is not true and so we print to screen that "The number is negative".

# Python Conditions and Elif statements

The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

```
# Check whether a number is positive, negative, or zero

x = 0  # Hardcoded value

# Decision making using if-elif-else
if x > 0:
    print("The number is positive.")
elif x == 0:
    print("The number is zero.")
else:
    print("The number is negative.")
```

```
if x > 0:
    print("The number is positive.")
```

- First, it checks if x is greater than 0.
- If **true**, it prints: "The number is positive."
- In this case, x = 0, so this condition is **false**.

```
elif x == 0:
    print("The number is zero.")
```

- This checks if x is **exactly equal to 0**.
- This condition is **true**, so it prints: "The number is zero."

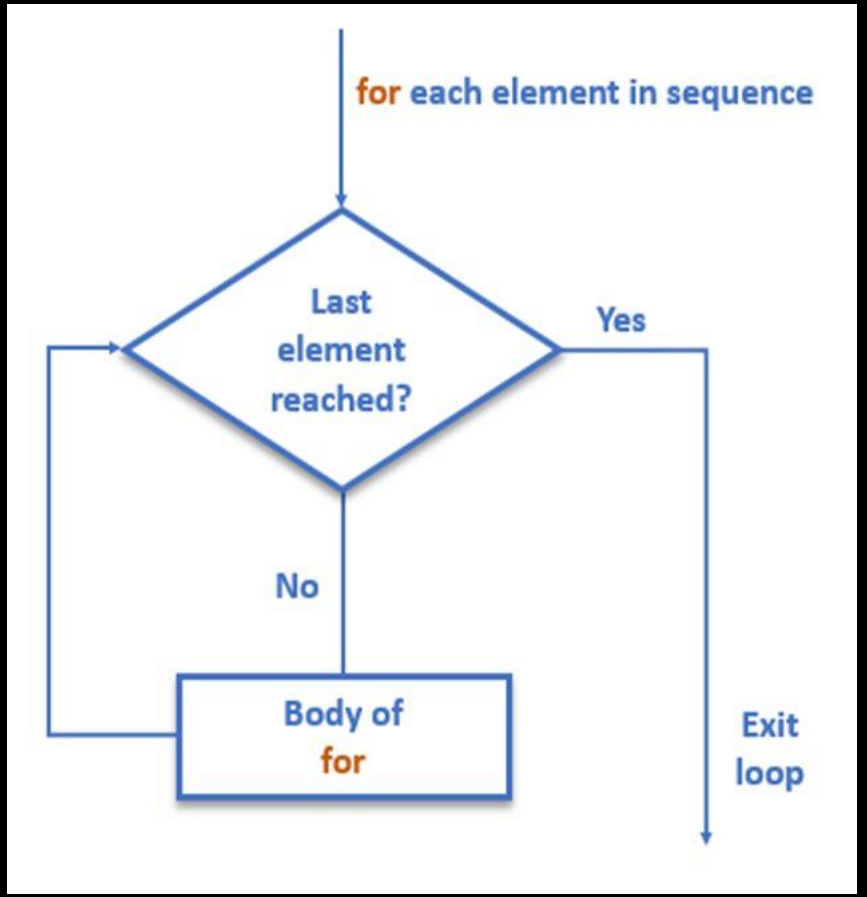
```
else:
    print("The number is negative.")
```

- The else part runs only if both previous conditions are false.
- Not executed here, because x == 0 matched.

# **Looping Statements** in Python

Used to **repeat a block of code** multiple times.

- for loop
  - Iterates over **a sequence** (list, string, tuple, dictionary, range).



# What is a for loop?

A for loop in Python is used to **iterate over a sequence** (like a list, tuple, string, dictionary, or a range). It is used to execute a block of code **multiple times**, once for each item in the sequence.

Basic Syntax

```
for variable in sequence:
    # code block
```

**Variable:** a temporary name that refers to the current element.
**Sequence:** a collection like list, tuple, string, etc.

Example 1: Loop Through a List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

# Break and Continue Statement

## What is break?

- break is used to **exit the loop immediately**, regardless of the loop condition.
- It is commonly used to **stop a loop when a condition is met.**

## What is continue?

- continue skips the **current iteration** and moves to the **next loop cycle.**
- Useful when you want to skip certain values but continue looping.

| | |
|---|---|
| Break Statement | • When you need to exit a loop early. |
| Continue Statement | • When you need to skip an iteration and continue to the next. |

```
print("\n--- For Loop Over a List ---")
fruits = ["apple", "banana", "cherry", "grape"]
for fruit in fruits:
    if fruit == "cherry":
        print("Found cherry, stopping early!")
        break  # Stops loop when cherry is found
    print(fruit)
```

```
print("\nList Loop with continue (skip 'banana'):")
for fruit in fruits:
    if fruit == "banana":
        continue  # Skip banana
    print(fruit)
```

# Python Function

## What is a Function?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code again and again, we define a function once and call it whenever needed.

## Why Use Functions?

- Code reuse
- Better readability
- Easier debugging
- Logical structure

Basic Syntax

```
def function_name(parameters):
    # code block
    return value
```

- def: Keyword to define a function.
- function_name: Name of your function.
- parameters: (optional) Values passed into the function.
- return: (optional) Sends result back to caller.

# Types of Function

- **Built-in Functions**: print(), len(), type(), etc.
- **User-defined Functions**: Functions you create using def.

**User-defined Functions**

- Function Without Parameters
- Function With Parameters
- Function With Multiple Parameters
- Default Parameters
- Function with Arbitrary Arguments – *args
  - Same Data Types
  - Mix Data Types
  - Combine *args with other parameters

# Function with Arbitrary Arguments – *args

## What is *args?

- *args allows a function to accept **any number of positional arguments.**
- The arguments are received as a **tuple.**
- Used when you're **not sure how many arguments** will be passed.

```
def function_name(*args):
    for item in args:
        # Do something
```

**Note:**

- *args is just a name by convention; you can use *numbers, *values, etc., but *args is recommended for clarity.

# Thank you!